정민혁 QA 포트폴리오

목차

PROFILE

경력 요약

회사 / 역할 / 기간 / 주요 업무와 성과

03

01

테스트 설계 및 QA 검증 방식

위험 기반 설계 / 검증 범위와 우선순위 / 수정·회귀 검증 / 릴리즈 판단

04 - 05

02

QA 프로세스 개선 및 커뮤니케이션

Slack QA Bot / STT 자막 도구 / AI 스펙 변경 추적

06 - 08

03

AI 제품 검증 및 테스트 자동화

평가 기준 수립 / Computer Use 에이전트 / 테스트 환경 자동화

09 - 11

경력 요약

스튜디오씨드코리아

2025.04 - 현재

QA Engineer / ProtoPie

- 웹 / 데스크톱 / 모바일 프로토타이핑 제품군과 기업용 환경의 E2E 및 릴리즈 / Hotfix 검증
- 인앱 AI 기능 QA
- 자동화 테스트 개발 및 테스트 실행과 결과 검토 환경 구축
- AI 활용 QA 프로세스 개선
- QA 업무용 슬랙 봇 개발

컴투스

2023.06 - 2024.06

QA Engineer / MLB 라이벌

- MLB 라이벌 Android / iOS / Steam 출시 및 라이브 서비스 QA
- 기능 / 밸런스 / 인게임 / 웹 이벤트 테스트 설계 및 실행, 플랫폼별 예외와 업데이트 회귀 검증
- 게임 플레이 데이터 처리 도구 개발: 약 2시간 → 3분 이내

엔씨소프트

2022.03 - 2022.12

Product Operations
프로야구 커뮤니티

- 사용자 데이터 기반 콘텐츠 기획 및 운영, 실시간 중계 퀴즈용 경기 데이터와 선수 기록 검수
- 게시글 / 댓글 / 스포츠 이벤트 분석 및 운영 데이터 수집과 정리 자동화: 약 2시간 → 10분

위험 기반 테스트 설계 기준

변경이 사용자 흐름과 기존 제품 동작에 미칠 위험을 먼저 분석하고, 중요도와 영향 범위를 기준으로 검증 범위와 테스트 구조를 정합니다.

01

요구사항과 기대 동작 사전 검토

TC / 체크리스트 작성 전 PRD / 디자인 / 개발 문서를 검토해 예상되는 위험과 기대 결과가 모호한 조건을 찾습니다. 기획 / 개발과 이를 논의해 검증 기준을 구체화합니다.

02

변경의 영향 범위와 검증 우선순위 판단

사용자 핵심 흐름의 중단, 기존 정책 / 동작과의 충돌, 연결된 제품에 미치는 영향을 우선 살핍니다. 변경된 기능뿐 아니라 영향을 받을 수 있는 제품과 기존 사용 흐름까지 검증 범위에 포함합니다.

03

위험을 기준으로 테스트 구조 설계

우선 검증할 사용자 흐름과 제품 간 연결 구간을 정하고, 계정 / 권한 / 구독 상태 등 필요한 조건을 나눕니다. 이 구조를 바탕으로 TC / 체크리스트와 회귀 테스트 범위를 구체화합니다.

검증 결과를 릴리즈 판단으로 연결

설계한 검증 범위와 조건에 따라 테스트를 수행하고, 수정 영향과 남은 리스크를 확인해 담당 QA 범위의 릴리즈 판단으로 연결합니다.

01 **변경·영향 범위 검토**
핵심 흐름과 연결 제품 기준으로 범위 / 우선순위 설정

02 **검증 흐름과 조건 구조화**
계정 / 권한 / 구독 상태와 허용 / 제한 동작 구분

03 **체크리스트 기반 실행**
우선 검증 흐름부터 수행 / 탐색·회귀·성능 검증

04 **수정 확인과 영향 회귀**
원래 조건과 영향을 받는 기존 사용 흐름 재검증

05 **결과와 잔여 리스크 공유**
개발 / 기획 공유 → 담당 QA 범위 Sign-off 판단

적용 예시: 저장 정책 변경 / 공개용 재구성

정상 저장 차단과 제한 우회를 우선 위험으로 두고, 파일 상태 / 저장 전후 용량 / 저장 방식으로 조건을 나눴습니다.

검증 조건	기대 결과
개수 제한을 넘는 기존 파일 보유 / Unlocked 파일 / 저장 후 용량 한도 이내	덮어쓰기 성공 / 새 Revision 생성 / 파일 개수와 Unlocked 상태 유지
Locked 파일에 덮어쓰기 시도	저장 차단 / 기존 Revision과 데이터 유지
Unlocked 파일 / 저장 후 용량 한도 초과	첫 저장부터 차단 / Revision 및 사용량 증가 없음 / 미저장 변경 유지

QA Bot: QA 업무를 돕는 Slack 에이전트 봇 개발

문제

- Daily report용 Jira 티켓을 우선순위 및 상태별로 수작업 집계
- 티켓 현황 정리에 약 30분 소요

판단과 실행

- 자연어 요청을 QA 도구와 API로 연결하는 Slack 에이전트 봇 직접 개발
- Jira 티켓을 우선순위와 상태별로 자동 집계하고 Daily report 생성
- Jira 티켓 번역 및 관리 자동화
- 개발용 AI Credit 지급 툴 연동

효과

- **수작업 티켓 집계에 약 30분 소요 → 자동 집계 결과를 1분 이내 확인**
- **QA 팀 전원 4명이 테스트마다 크레딧 관리와 테스트 기기 세팅에 활용**

Daily report / Jira 티켓 현황 자동 집계

버그 현황 (최근 14일 생성분) — 17건 · 신규 24h 0건

Resolved = Ready for QA ~ Done(개발 수정 완료 이후) · Unresolved = 그 외 상태

[redacted] (PS)

Total	Resolved	Unresolved
6	3	3

QA용 Credit 관리 연동 / 구성도

요청
Slack에서 조건 선택
Owner / Team 등 요청 조건 입력

연동
Credit 관리 툴 실행
QA Bot에서 조회 / 지급 작업 연결

버그 재현 영상 자막 생성 도구 개발

문제

- 웹과 앱을 함께 쓰는 제품 특성상 복잡한 버그 재현 과정
- 소리 없이 영상을 보는 개발자에게 녹음 설명이 전달되지 않는 문제

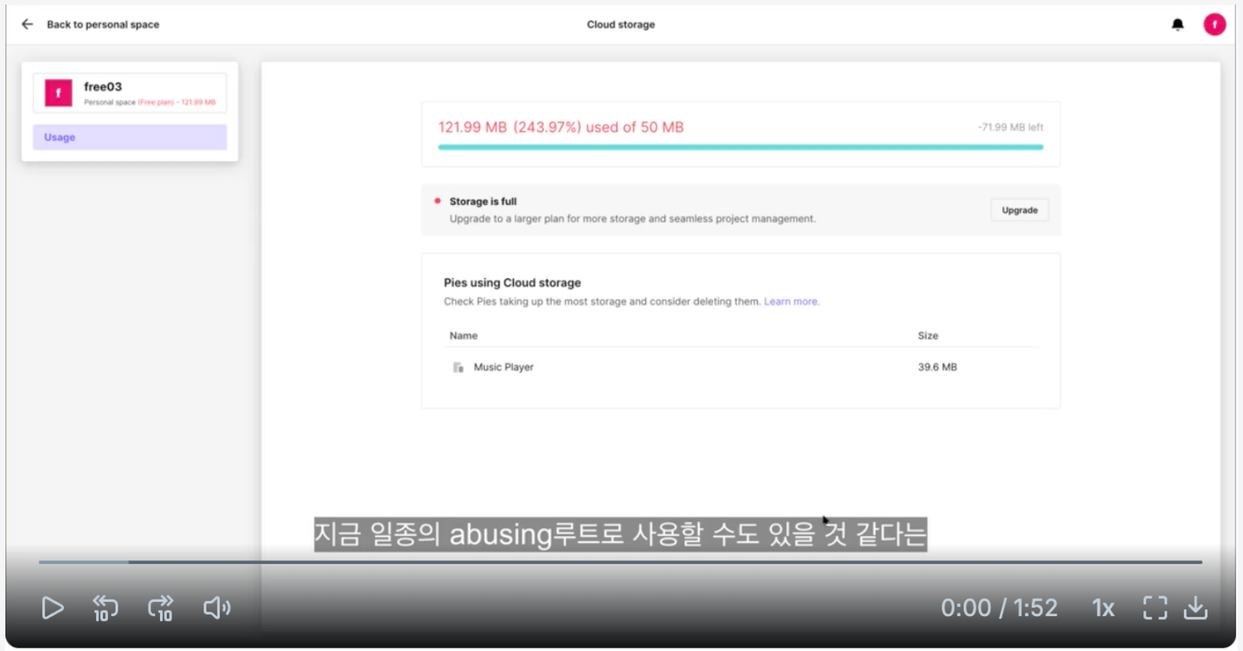
판단과 실행

- 복잡한 재현 과정을 직접 녹음해 설명
- 오픈소스 Whisper STT 기반 자막 프로그램을 개발해 활용
- 녹음 설명을 자막으로 변환해 무음 시청에서도 내용 전달

효과

- **무음 시청에서도 복잡한 재현 절차와 설명을 함께 확인 가능**
- **적용 후 설명 부족으로 인한 재설명 요청 / 재현 불가 문의 감소**

Jira에 첨부한 자막 포함 재현 영상



AI를 활용한 QA 프로세스 개선

문제

- 빨라진 개발 속도와 잦은 스펙 변경에 맞춰 변경 이력 추적 필요
- 여러 업무 툴에 흩어진 논의와 결정사항을 수작업으로 확인

판단과 실행

- AI로 여러 업무 툴의 논의 및 결정사항 수집, 변경 맥락 정리
- 제품 스펙 / Test Design / Checklist 업데이트 제안
- QA가 제안 내용을 최종 검증하고 반영 승인
- 승인 전에는 변경 제안만 생성하고 원본 수정 제한

효과

- **분산된 논의 및 변경사항을 스펙 / Test Design 변경 제안으로 정리**
- **QA가 최종 검토하고 승인한 변경사항만 반영하도록 통제**

AI 제안과 QA 승인 프로세스

01 / AI — **결정사항 수집 & 정리**
Notion / Figma / Jira / Slack

02 / AI — **스펙 / Test Design 변경 제안**
제품 스펙 / Test Design / Checklist

03 / QA — **내용 검증 & 승인**
승인 전에는 원본 수정 제한

04 / 반영 — **승인된 변경 반영**
검증 기준을 최신 제품 맥락에 맞춰 유지

실사용 패턴으로 AI 평가 기준 구체화

문제

- 디자이너를 위한 인앱 AI 기능의 평가 기준이 모호

판단과 실행

- 디자인팀과 협업하여 자주 사용하는 제품 활용 패턴 선정
- 실사용 패턴을 평가용 TC로 구성
- 패턴 완성도를 기준으로 모델별 결과 평가
- 동일 프롬프트를 반복 입력해 응답 품질과 재현성, 안정성 평가

효과

- **디자이너의 실사용 패턴 완성도를 AI 결과 평가의 공통 기준으로 적용**

실사용 패턴 기반 평가 기준 수립

01 / 문제	**AI 기능의 평가 기준이 모호** 디자이너의 사용 맥락을 반영할 기준 필요
02 / 협업	**디자인팀과 실사용 패턴 선정** 제품에서 자주 사용하는 패턴을 중심으로 선정
03 / 설계	**패턴을 평가용 TC로 구성** 같은 사용 목적을 기준으로 모델별 결과 비교
04 / 평가	**완성도 및 반복 결과 평가** 동일 프롬프트 반복으로 품질 / 재현성 / 안정성 확인

Computer Use 기반 QA 에이전트 개발 & 활용

문제

- OS / 웹 / 앱을 넘나드는 실제 사용자 행동의 자동화 필요
- 에이전트 실행 결과의 실제 성공 여부를 확인할 검토 구조 필요

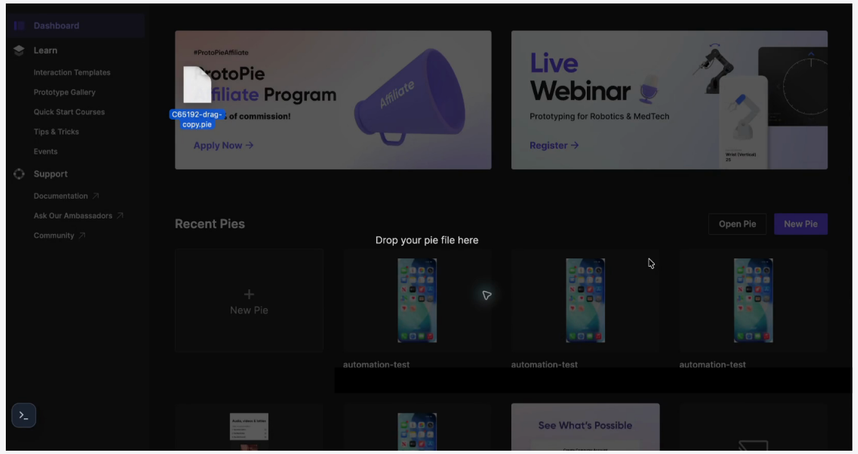
판단과 실행

- Computer Use로 TC를 수행하는 QA 에이전트를 개발해 활용
- 제품별 지식 DB 구축 및 사람용 / 에이전트용 TC 1:1 관리
- TC별 실행 과정과 근거를 기록하는 대시보드 구성
- 모델 특성에 맞춘 하네스 조정으로 클릭 정확성과 속도 개선

효과

- **OS / 웹 / 앱을 넘나드는 TC 실행 자동화**
- **TC별 실행 과정과 근거를 한 화면에서 확인해 최종 결과 검토**

Computer Use / 테스트 파일을 데스크톱 앱으로 드래그&드롭



TC 실행 근거와 최종 결과 검토 / 구성도

TC별 실행 과정과 근거

대시보드에 실행 기록 정리

사람이 성공 여부 확인

관찰 기록과 최종 판정 구분

테스트 기기 QA 환경 세팅 자동화

문제

- macOS / Windows에 Desktop App / Embedded 빌드를 각각 수동 설치
- Windows의 긴 설치 시간을 포함해 테스트 전 빌드 준비에 약 30분 소요

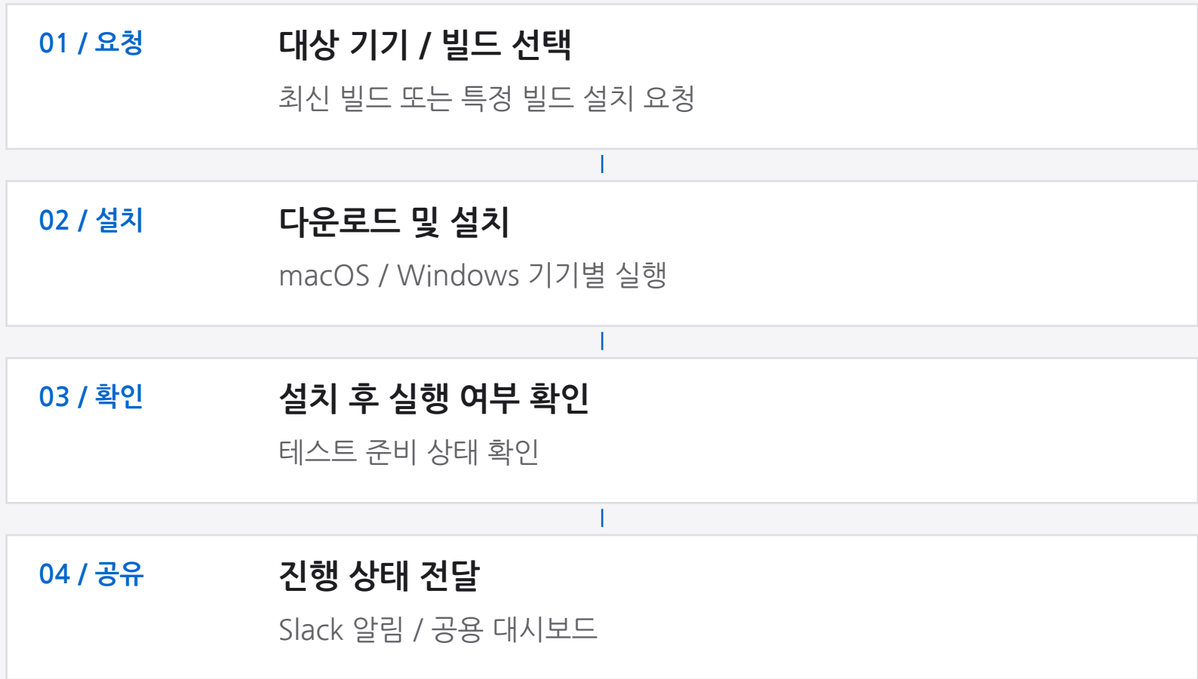
판단과 실행

- 접근을 Tailscale 내부망으로 제한하고 SSH로 기기별 설치 스크립트 실행
- 테스트 대상 빌드 자동 다운로드 및 설치 후 실행 여부 확인
- 최신 빌드 자동 업데이트 및 특정 빌드 설치 요청 지원
- 기기별 설치 진행 상태를 Slack으로 전달

효과

- **macOS / Windows 빌드 준비에 약 30분이 들던 수동 작업 자동화**
- **QA 팀 전원이 테스트마다 활용**

테스트 환경 준비와 진행 상태 공유 / 구성도



연락처

정민혁

QA Engineer

tlsfkaus4862@naver.com

010-5535-9426

GitHub / Blog

학력 및 자격

성공회대학교

경영학 / 소프트웨어공학 복수전공 학사

SQLD